# LAPORAN PRAKTIKUM
# PEMROGRAMAN BERORIENTASI OBJEK
# MODUL 05 – RELASI ANTAR KELAS




Jury Maxwell

103132400023

PS1DS-05-REG-01

**PROGRAM STUDI SAINS DATA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

## A. Dasar Teori

Relasi antar kelas dalam Pemrograman Berorientasi Objek (OOP) memodelkan bagaimana entitas-entitas kelas saling berinteraksi, bergantung, atau membentuk struktur sistem yang lebih kompleks. Diagram kelas (Class Diagram) digunakan untuk memvisualisasikan kelas-kelas yang terlibat dalam sistem beserta relasi keterhubungannya, di mana notasi garis tertentu digunakan untuk membedakan derajat keeratan ikatan antar objek.

1. **Asosiasi:** Merupakan hubungan logis paling longgar antara dua kelas yang berdiri sendiri ("uses-a"). Objek dari kelas pertama berinteraksi dengan objek kelas kedua biasanya melalui parameter pada suatu method atau pemanggilan fungsi, tanpa menjadikan objek tersebut sebagai atribut field permanen. Siklus hidup kedua objek sepenuhnya independen dan terpisah.
2. **Agregasi:** Merupakan hubungan struktural bertingkat yang menyatakan hubungan bagian dan keseluruhan ("has-a" / whole-part). Sebuah kelas wadah (whole) menyimpan referensi satu atau lebih objek lain (part) sebagai atribut. Namun, objek part diciptakan secara independen di luar wadah dan dapat terus hidup meskipun objek wadah dihancurkan.
3. **Komposisi:** Merupakan bentuk khusus dari agregasi dengan derajat kepemilikan yang sangat ketat ("strong has-a" / death relationship). Objek bagian (part) diciptakan langsung di dalam kelas wadah (whole) dan siklus hidupnya terikat mutlak pada wadah tersebut. Jika objek whole musnah, maka seluruh objek part di dalamnya ikut musnah dan tidak dapat digunakan lagi secara mandiri.

## B. Guided

### 1. Asosiasi

File: Mahasiswa.java

```java
package guided.asosiasi;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama){
        this.nama = nama;
    }

    public void setNim(String nim){
        this.nim = nim;
    }

    public void setNilai(int nilai){
```

```
        this.nilai = nilai;
    }

    public int getNilai(){
        return this.nilai;
    }

    public String getNama(){
        return this.nama;
    }

    public String getNim(){
        return this.nim;
    }
}
```

File: Dosen.java

```
package guided.asosiasi;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    public void setKodeDosen(String kodeDosen){
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen){
        this.namaDosen = namaDosen;
    }

    public String getKodeDosen(){
        return this.kodeDosen;
    }

    public String getNamaDosen(){
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai){
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s){
        return s.getNilai();
    }
```

```
    public static void main(String[] args){
        Mahasiswa m = new Mahasiswa();

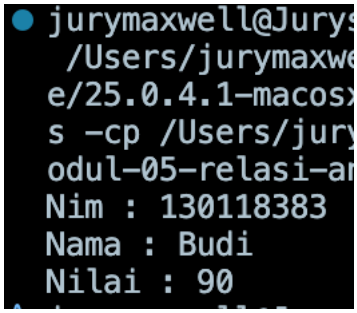
        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim : " + m.getNim());
        System.out.println("Nama : " + m.getNama());
        System.out.println("Nilai : " + d.getScore(m));
    }

}
```

**Output :**




2. **Agregasi**

File: Mahasiswa.java

```
package guided.agregasi;

public class Mahasiswa {
    private String NIM;
    private String Nama;

    public Mahasiswa (String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    public String getNim(){
        return (NIM) ;
    }

    public String getNama(){
        return (Nama) ;
    }
```

```
    public void setNim(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }

}
```

File: Jurusan.java

```
package guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i = 0; i<jumlahMahasiswa; i++){
            System.out.println(i+1+ ". " +
            daftarMahasiswa[i].getNim() + "  "
            + daftarMahasiswa[i].getNama());
        }
    }

    //getter
    public String getKodeJurusan(){
```

```
        return kodeJurusan;
    }

    public String getNamaJurusan(){
        return namaJurusan;
    }

    //setter
    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }


}
```

3. **Komposisi**

File: Processor.java

```
package guided.komposisi;

public class Processor {
    private String brand;
    private int Kecepatan;

    public Processor(String brand, int kecepatan){
        this.brand = brand;
        this.Kecepatan = kecepatan;
    }

    //behavior
    public void displaySpec(){
        System.out.println(this.brand + " " + this.Kecepatan);

    }
}
```

File: Monitor.java

```
package guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
```

```
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }

}
```

File: Mouse.java

```
package guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }

}
```

File: Komputer.java

```
package guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
```

```
    }
}
```

File: Main.java

```
import guided.agregasi.*;
import guided.komposisi.*;

public class Main {

    public static void main(String[] args) {
        runAgregasi();
        runKomposisi();
    }

    public static void runAgregasi(){
        Mahasiswa m1 = new Mahasiswa("001", "Max");
        Mahasiswa m2 = new Mahasiswa("002", "Min");

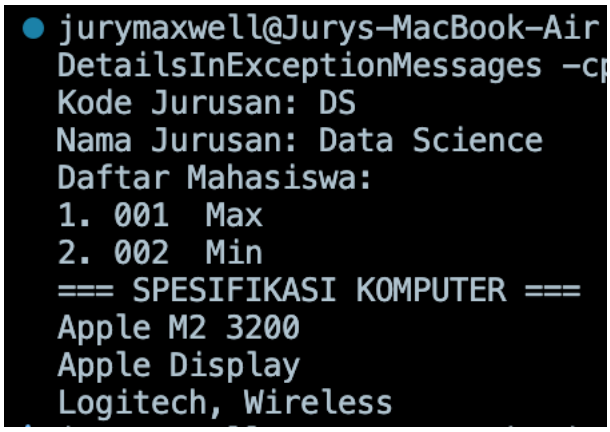
        Jurusan prodi = new Jurusan("DS", "Data Science");
        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);
        prodi.displayMahasiswa();
    }

    public static void runKomposisi(){
        Komputer mac = new Komputer("Apple M2", 3200, "Apple Display",
"Logitech", "Wireless");
        mac.displaySpecKomputer();

    }


}
```

**Output :**




## C. Unguided

### 1. Soal Unguided 1

**Soal — Sistem Manajemen Pesanan Cafe**

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

**Tugas 1: Implementasi Komposisi (Composition)**

Komposisi adalah relasi di mana sebuah objek memiliki objek lain sebagai bagian internalnya yang tidak dapat berdiri sendiri. Siklus hidup objek internal terikat penuh pada objek utamanya.

Pada sistem cafe:

- Setiap item pesanan (ItemPesanan) pasti memiliki rincian harga (RincianHarga) yang mencatat harga dasar dan pajak (10%).
- Objek RincianHarga harus di-instansiasi langsung di dalam constructor ItemPesanan. Objek RincianHarga tidak pernah dibuat di main. Jika ItemPesanan dihapus, maka RincianHarga miliknya ikut hilang.

**Kebutuhan Tugas 1**

Buatlah dua class: RincianHarga dan ItemPesanan.

**Field yang disarankan:**

| Class | Field | Jenis | Tipe | Keterangan |
|---|---|---|---|---|
| RincianHarga | hargaDasar | Instance field | double | Harga awal sebelum pajak |
| RincianHarga | pajak | Instance field | double | Nilai nominal pajak (10% dari harga dasar) |

| | | | | |
|---|---|---|---|---|
| ItemPesanan | namaItem | Instance field | String | Nama menu (misal: "Iced Americano") |
| ItemPesanan | rincian | Instance field | RincianHarga | Objek internal rincian harga (Komposisi) |

**Method yang disarankan:**

| Class | Method | Return | Keterangan |
|---|---|---|---|
| RincianHarga | getHargaTotal() | double | Mengembalikan nilai hargaDasar + pajak |
| ItemPesanan | getNamaItem() | String | Mengembalikan nama item pesanan |
| ItemPesanan | getRincian() | RincianHarga | Mengembalikan objek rincian harga |

### Tugas 2: Implementasi Agregasi (Aggregation)

Agregasi adalah relasi di mana sebuah objek menampung kumpulan objek lain yang dapat berdiri sendiri. Objek yang ditampung dibuat secara independen di luar, lalu dimasukkan ke dalam objek wadah.

Pada sistem cafe:

- Sebuah Pesanan menampung beberapa objek ItemPesanan menggunakan array standar.
- Objek ItemPesanan dibuat terpisah terlebih dahulu (di main), lalu dimasukkan satu per satu ke dalam Pesanan melalui method tambahItem(). Jika Pesanan dibatalkan atau dihapus, objek ItemPesanan tersebut secara konsep tetap ada.

**Kebutuhan Tugas 2**

Buatlah class Pesanan.

**Field yang disarankan:**

| Field | Jenis | Tipe | Keterangan |
|---|---|---|---|
| nomorMeja | Instance field | int | Nomor meja pelanggan |
| daftarItem | Instance field | ItemPesanan[] | Array penampung item pesanan (Agregasi) |
| jumlahItem | Instance field | int | Penanda jumlah item yang sudah masuk ke array |

**Method yang disarankan:**

| Method | Return | Keterangan |
|---|---|---|
| tambahItem(ItemPesanan item) | void | Memasukkan objek ItemPesanan dari luar ke dalam array daftarItem |

| | | |
|---|---|---|
| tampilkanPesanan() | void | Menampilkan nomor meja dan seluruh item beserta harga total per item |
| getNomorMeja() | int | Mengembalikan nomor meja |
| getDaftarItem() | ItemPesanan[] | Mengembalikan array daftar item |
| getJumlahItem() | int | Mengembalikan jumlah item yang tersimpan |

**Tugas 3: Implementasi Asosiasi (Association)**

Asosiasi adalah relasi interaksi antara dua objek independen yang saling menggunakan tetapi tidak saling memiliki. Objek tidak menyimpan objek lain sebagai field permanen, melainkan berinteraksi melalui parameter method.

Pada sistem cafe:

- Seorang Kasir memproses pembayaran dari sebuah Pesanan.
- Class Kasir tidak menyimpan objek Pesanan sebagai field, melainkan menerima objek Pesanan sebagai parameter pada method prosesPembayaran().

**Kebutuhan Tugas 3**

Buatlah class Kasir.

**Field yang disarankan:**

| Field | Jenis | Tipe | Keterangan |
|---|---|---|---|
| namaKasir | Instance field | String | Nama petugas kasir |

**Method yang disarankan:**

| Method | Return | Keterangan |
|---|---|---|
| prosesPembayaran(Pesanan pesanan) | void | Mengakses data Pesanan, menghitung akumulasi total bayar seluruh item, lalu mencetak struk pembayaran (Asosiasi) |

**Tugas 4: Eksekusi dan Pengujian di Main**

Buatlah class Main untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.

**Alur program di main**

1. Buat 1 objek Kasir (misal: "Budi").
2. Buat 3 objek ItemPesanan secara independen:

- "Iced Americano", harga dasar: 18000
- "Beef Croissant", harga dasar: 25000
- "Matcha Latte", harga dasar: 22000

3. Buat 1 objek Pesanan untuk Meja 4 (kapasitas array misal: 5 item).

4. Masukkan ketiga objek ItemPesanan ke dalam objek Pesanan menggunakan method tambahItem().

5. Tampilkan rincian pesanan dengan memanggil tampilkanPesanan() pada objek Pesanan.

6. Jalankan pemrosesan pembayaran dengan memanggil method prosesPembayaran(pesanan) dari objek Kasir.

**Jawaban :**

File RincianHarga.java:

```java
package unguided;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = (hargaDasar < 0) ? 0 : hargaDasar;
        this.pajak = this.hargaDasar * 0.10;
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

File ItemPesanan.java:

```java
package unguided;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
```

```
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

File Pesanan.java:

```
package unguided;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        } else {
            System.out.println("Kapasitas pesanan meja " + this.nomorMeja
+ " penuh!");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
            System.out.println((i + 1) + ". " + item.getNamaItem() + " -
Rp " + item.getRincian().getHargaTotal() +
                    " (Harga: " + item.getRincian().getHargaDasar() + ",
Pajak: " + item.getRincian().getPajak() + ")");
        }
```

```
    }

    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }
}
```

File Kasir.java:

```
package unguided;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] items = pesanan.getDaftarItem();
        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            totalSubtotal += items[i].getRincian().getHargaDasar();
            totalPajak += items[i].getRincian().getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " +
pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " +
pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
```

```
        System.out.println("============================");
    }

    public String getNamaKasir() {
        return this.namaKasir;
    }
}
```

File Main.java:

```
package unguided;

public class Main {
    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");

        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        Pesanan pesanan = new Pesanan(4, 5);
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        pesanan.tampilkanPesanan();
        System.out.println();
        kasir.prosesPembayaran(pesanan);
    }
}
```

**Output :**

```
jurymaxwell@Jurys-MacBook-Air modul-05-relasi-antar-kelas %  c
scode/extensions/redhat.java-1.57.0-darwin-arm64/jre/25.0.4.1-r
m/modul-05-relasi-antar-kelas/project/target/classes unguided.M
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

**Penjelasan :**

Program sistem kasir cafe ini dikembangkan untuk memodelkan proses bisnis operasional cafe secara modular melalui penerapan tiga relasi fundamental dalam Pemrograman Berorientasi Objek. Sistem ini memfasilitasi pencatatan data item menu pesanan beserta kalkulasi pajaknya, pengelompokan item pesanan ke dalam wadah transaksi per meja, hingga kalkulasi tagihan akhir dan pencetakan struk pembayaran oleh kasir yang bertugas.

Pada struktur internalnya, relasi Komposisi diterapkan antara kelas ItemPesanan dan RincianHarga. Di dalam constructor ItemPesanan, objek RincianHarga langsung diinstansiasi menggunakan kata kunci new sehingga siklus hidup rincian harga terikat mati pada keberadaan item pesanan dan perhitungan pajak sepuluh persen terenkapsulasi dengan aman. Selanjutnya, relasi Agregasi diwujudkan antara kelas Pesanan dan ItemPesanan, di mana kelas Pesanan memiliki atribut wadah berupa array ItemPesanan bertipe tetap yang menerima objek-objek item pesanan yang telah dibuat secara independen di kelas Main melalui pemanggilan method tambahItem.

Sementara itu, relasi Asosiasi direalisasikan pada interaksi antara kelas Kasir dan kelas Pesanan. Kelas Kasir tidak menyimpan referensi objek Pesanan sebagai atribut permanen dalam strukturnya, melainkan hanya menerima objek Pesanan sebagai parameter temporer pada method prosesPembayaran. Melalui method tersebut, Kasir mengakumulasikan total harga dasar dan total pajak dari seluruh item yang tersimpan di dalam pesanan, lalu mencetak struk transaksi resmi ke layar konsol secara fungsional tanpa menimbulkan dependensi struktural yang mengikat.

## D. Kesimpulan

Pada praktikum Modul 5 ini, saya mempelajari konsep fundamental Relasi Antar Kelas dalam Pemrograman Berorientasi Objek untuk membangun arsitektur perangkat lunak yang

modular, terstruktur, dan terenkapsulasi dengan baik. Melalui implementasi sistem perangkat keras komputer dan sistem kasir cafe, saya memahami secara mendalam perbedaan karakteristik antara Asosiasi (relasi ketergantungan fungsional temporer melalui parameter method), Agregasi (relasi struktural wadah dan isi di mana siklus hidup objek bagian tetap independen), serta Komposisi (relasi kepemilikan eksistensial mutlak di mana objek bagian diciptakan di dalam wadah dan siklus hidupnya terikat penuh pada objek induk).

Pemilihan jenis relasi yang tepat sesuai domain masalah nyata terbukti sangat krusial dalam rekayasa perangkat lunak, karena mampu mengontrol visibilitas data, mengeliminasi duplikasi logika, mencegah kebocoran referensi internal objek, serta memastikan sistem mudah dirawat (maintainable) dan dikembangkan (scalable) di masa mendatang.